Laborationsrapport

Projektuppgift
DT071G, Programmering i C#.NET

Ludvig **Rosenqvist,** **luro2501@student.miun.se**
HT, 2026



Mittuniversitetet
MID SWEDEN UNIVERSITY

1 Sammanfattning

Denna rapport dokumenterar utvecklingen av en interaktiv konsolapplikation i C# och .NET. Syftet med projektet har varit att tillämpa grundläggande och avancerade principer inom objektorienterad programmering, med ett särskilt fokus på att skapa en välstrukturerad, skalbar och lättunderhållen kodbas.

Applikationen är uppbyggd enligt SOLID-principerna och separerar koden inom olika delar för ansvarsområden, logik och användargränssnitt. För hantering av speltillstånd och datalagring används JSON-serialisering via biblioteket System.Text.Json. Användargränssnittet i terminalen har byggts med det externa biblioteket Spectre.Console för att erbjuda en visuellt tilltalande och tangentbordsnavigerad användarupplevelse. Vidare nyttjas LINQ (Language Integrated Query) för effektiv filtrering och manipulering av datasamlingar, samt WebUtility/HttpUtility för sanering och avkodning av HTML-entiteter.

Resultatet är en stabil, plattformsoberoende konsolapplikation med hög kodkvalitet, tydlig felhantering och modulär struktur. Projektet visar hur tillämpningen av etablerade principer och .NET-plattformens inbyggda ramverk minskar kodens komplexitet och underlättar framtida vidareutveckling.

2 Innehållsförteckning

1 Sammanfattning 1
2 Innehållsförteckning 2
3 Inledning 3
3.1 Syfte 4
3.2 Problemformulering 4
4 Teori 4
4.1 Programmeringsspråket C# och .NET-plattformen 5
4.2 SOLID principer 5
4.3 Datalagring och Serialisering 5
4.4 Konsolgränssnitt och Externa Ramverk 5
4.5 Datamanipulering och Frågespråk 6
4.6 Textsanering 6
5 Metod 6
5.1 Arbetsprocess & struktur 7
5.2 Programvaror & verktyg 7
5.3 Designprinciper & kodstandard 7
6 Konstruktion 8
6.1 Arkitektur & filstruktur 9
6.2 Datalagring och gränssnitt 9
6.3 Spelflöde & logik 10
6.4 Användargränssnitt och navigation 11
7 Resultat 12
8 Slutsatser 13
8.1 Reflektion kring resultat 14
8.2 Lärdomar och utveckling 14
8.3 Framtida utveckling 14
9 Källförteckning 15

3 Inledning

3.1 Syfte

Syftet med detta projekt är att skapa en fullt fungerande, estetiskt tilltalande och användarvänlig konsolapplikation för ett quizspel. Applikationen ska erbjuda användaren val av kategorier och svårighetsgrad, beräkna poäng i realtid och låta användaren spara sina poäng även efter applikationen avslutas. Projektet ska även anta en modulär kodstruktur med etablerade designmönster och objektorienterade principer inom C# och .NET.

3.2 Problemformulering

Vid utveckling av konsolbaserade applikationer är det lätt att logik, användargränssnitt och datahantering sammanflätas tillsammans i endast en fil, exempelvis i Program.cs. Detta medför att koden snabbt blir komplicerad svårunderhållen och svår att bygga vidare på. För att säkerställa att detta projekt utvecklas med hög kvalité och struktur har projektet anpassats för att lösa följande problemställningar:

- **Modularitet:** Hur kan kodstrukturen delas upp för att separera ansvaret i distinkta moduler så att förändringar i användargränssnittet eller datakällan inte kräver omfattande förändringar i kodens logik?

- **Användarupplevelse:** Hur kan ett terminalbaserat gränssnitt lyftas från manuella textinmatningar till en dynamisk och engagerande användarupplevelse med tangentbordsnavigering och tydligare uppvisning av data?

- **Extern datahantering:** Hur kan data från en extern källa filtreras, sorteras och saneras från oönskade specialtecken innan de presenteras för användaren?

- **Spelflöde och validering:** Hur kan applikationens flöde undvika krascher eller buggar under den aktiva sessionen och hur kan datatyper valideras innan körning för att hålla applikationen robust och säker?

4 Teori

4.1 Programmeringsspråket C# och .NET-plattformen

C# är ett programmeringsspråk med öppen källkod utvecklat av Microsoft. Språket är plattformsoberoende vilket gör det oerhört brett och anpassningsbart för många olika typer av program som exempelvis mobilapplikationer eller konsolapplikationer med .NET-plattformen. C# ingår i C-språkfamiljen vilket gör syntaxen bekant för utvecklar som kommer från en bakgrund i C, C++, JavaScript, TypeScript eller Java [1].

4.2 SOLID principer

SOLID principerna består av fem centrala riktlinjer för att göra kod lättare att underhålla och bygga vidare på.

- **Single Responsibility (S):** En klass ska endast ha ett ansvarsområde eller syfte.
- **Open-Closed Principle (O):** Varje klass ska vara öppen för utbyggnad men stängd för modifikation för att undvika att kod som använder klassen blir okörbar.
- **Liskov's Substitution Principle (L):** Varje klass som ärvt egenskaper måste kunna ersätta sin förälder utan oförväntade beteenden i koden.
- **Interface Segregation Principle:** Ett gränssnitt (interface) ska fokusera på specifika och väldefinierade ansvarsområden.
- **Dependency Inversion Principle:** Stora moduler ska inte vara beroende ut av mindre moduler. Istället ska ett abstraktionslager i form av ett interface binda dem samman för att undvika hårdkodning.

[2]

4.3 Datalagring och Serialisering

JSON är ett textbaserat dataformat för att lagra och byta ut data på ett sätt som är läsbart för både människor och maskiner. JSON kommer ursprungligen från JavaScript men har vuxit långt utanför det språket och blivit ett populärt dataformat för att hantera data mellan flera olika typer av plattformar och programmeringsspråk.

För att hantera detta i C# och .NET används det inbyggda biblioteket *System.Text.Json*. Detta bibliotek tillåter serialisering och deserialisering för att omvandla C#-objekt till JSON-strängar och vice versa [3].

4.4 Konsolgränssnitt och Externa Ramverk

För att kommunicera med exempelvis datorer, servrar och applikationer används CLI (Command Line Interface) för att skriva kommandon direkt i en terminal [4].

Detta kommunikationssätt kan utökas med externa bibliotek, såsom .NET-biblioteket Spectre.Console, för att skapa mer tilltalande användargränssnitt direkt i terminalen. Detta möjliggör funktioner som att rita upp detaljerade tabeller och rutnät, samt låta användaren navigera i gränssnittet interaktivt med tangentbordet via färdiga menyalternativ [5].

4.5 Datamanipulering och Frågespråk

LINQ (Language Integrated Query) är funktionalitet som tillåter utvecklare att manipulera data genom användning av användbara och mycket läsliga funktioner. Likt SQL-språk kan funktioner som Where(), Select() och OrderBy() användas direkt på C#-element.

Tekniken stödjer två huvudsakliga skrivsätt: frågesyntax (som liknar SQL-uttryck med nyckelord som from, where och select) och metodsyntax. Metodsyntaxen ger fullständig tillgång till alla standardoperatorer, däribland filtrering (Where), transformering (Select), sortering (OrderBy) samt aggregering och gruppering (GroupBy). Detta minskar behovet av manuella loopar och villkorssatser, vilket gör koden både säkrare vid kompilering och betydligt lättare att underhålla [6].

4.6 Textsanering

När data hämtas från externa källor, API:er eller filer kan texten ibland innehålla HTML-entiteter eller specialtecken (exempelvis att tecken som < och > kodas om till < och > för säker överföring). För att säkerställa att dessa tecken tolkas korrekt och presenteras snyggt i terminalen används avkodning.

I .NET kan detta hanteras genom metoder som HttpUtility.HtmlDecode, vilket omvandlar HTML-kodade strängar tillbaka till vanliga, läsbara tecken. Detta säkerställer att presentationen i konsolapplikationen inte störs av kodade tecken eller specialsymboler [7].

5 Metod

5.1 Arbetsprocess & struktur

Utveckling inleddes med en planeringsfas där applikationens grundläggande krav och flöden definierades. Arbetet delades därefter upp i tre huvudsakliga faser:

1. **Datatyper:** Första steget var att bygga upp de datastrukturer som applikationen krävde, vilket lade grunden för hantering av både frågor (Question) och sparade resultat (HighScore).

2. **Kärnlogik och datahantering:** Därefter implementerades logiken för att läsa in data, hantera spelsessionen och spara ner high scores till fil via JSON.

3. **Gränssnitt och navigation:** Den sista fasen fokuserade på att bygga upp det interaktiva navigationssystemet. Varje skärm delades upp i egna moduler och formaterades med hjälp av det externa biblioteket Spectre.Console för att ge en ren och tydlig upplevelse direkt i terminalen.

5.2 Programvaror & verktyg

Följande programvaror och verktyg har använts under projektets gång:

- **Programspråk och plattform:** Applikationen utvecklades med C# och .NET med programmet Visual Studio.

- **Externa bibliotek:**

    - **Spectre.Console:** Används för att skapa ett modernare och mer användarvänligt gränssnitt gentemot en klassisk konsolapplikation. Biblioteket tillät skapandet av formaterade tabeller, rubriker, färgad text och multivals-inputs.

    - **System.Text.Json:** Ett bibliotek inbyggt i .NET som används för att serialisera och deserialisera JSON-data vid sparande och läsning av externa JSON-filer.

- **Versionshantering:** Git och GitHub har använts för källkods- och versionshantering för att säkerställa att programmet inte skrivs över och blir okörbart.

5.3 Designprinciper & kodstandard

Projektet har genomgående följt SOLID-principerna och objektorienterad programmering för att säkerställa hög kvalité och läsbarhet:

- **Separation av ansvar (Separation of Concerns):** För att undvika att all logik hamnade i en enda fil delades koden upp i distinkta klasser med avgränsade ansvarsområden. Exempelvis separerades logiken från gränssnittet i moduler som GameSession, GameScreen och QuizService.

- **Dependency Injection:** För att moduler inte skulle hårdkodas ihop skapades gränssnitten IQuestionLoader och IHighScoreStorage. Detta gjorde att systemets logik kunde byggas mot gränssnittet istället för specifika filtyper. Om exempelvis JSON-lagringen behöver bytas ut mot en SQL-databas i framtiden kan det göras enkelt utan att koden rasar och blir okörbar.

- **Läsbarhet och metoddesign:** Metoderna har hållits korta, fokuserade och självförklarande. Istället för att fylla filerna med inline-kommentarer har koden gjorts lättläst genom att använda tydliga namn på både metoder och variabler. Även hur koden körs är designad för att undvika för mycket nestning vilket gör koden svårläst.

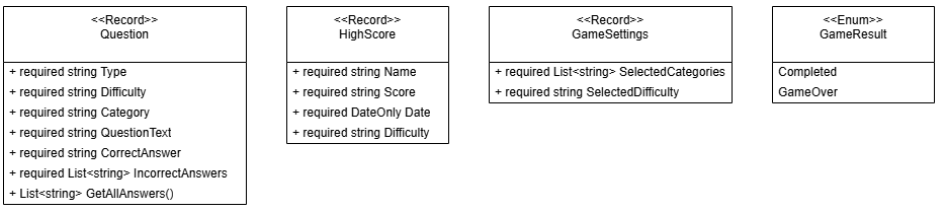
6 Konstruktion

6.1 Arkitektur & filstruktur

Applikationens filer är uppdelade i mappar baserat på deras funktionalitet och ansvarsområde. Följande mappstruktur implementerades i projektet:

- **Models** – Datastrukturer och modeller
- **Interfaces** – Gränssnitt
- **Services** – Kärnlogik och datahantering
- **Screens** – Användargränssnitt
- **Utils** – Användbara funktioner

Ett antal klasser skapades för de datatyper som behövdes för att uppnå projektets mål. Till en början bestod dessa av vanliga klasser, men vidareutvecklades senare under projektets gång till att använda mer moderna datatyper som record och enum. Samtliga datatyper och deras relationer illustreras i klassdiagrammet nedan **(se Figur 1)**.

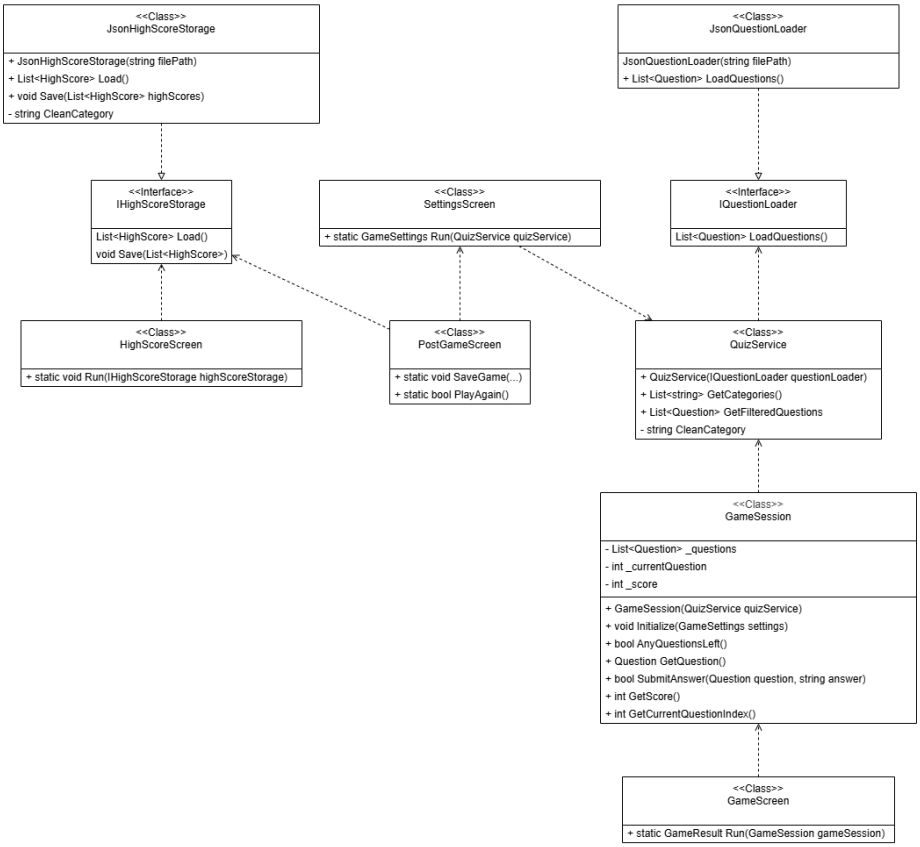


Figur 1: Klassdiagram som innehåller de skapade datatyperna.

6.2 Datalagring och gränssnitt

För att sammankoppla applikationens datatyper med logik implementerades gränssnitt för att tillämpa *Dependency Injection* för att säkerställa att applikationens delar inte hårdkodas mot varandra.

I klassdiagrammet nedan **(se Figur 2)** illustreras hur datatyperna, gränssnitten och tjänsterna relaterar till varandra inom applikationen.



Figur 2: Klassdiagram som visar relationerna mellan applikationens datatyper, gränssnitt och tjänster.

6.2.1 Inläsning och sparande av data

För att hantera dataläsning och lagring skapades två gränssnitt:

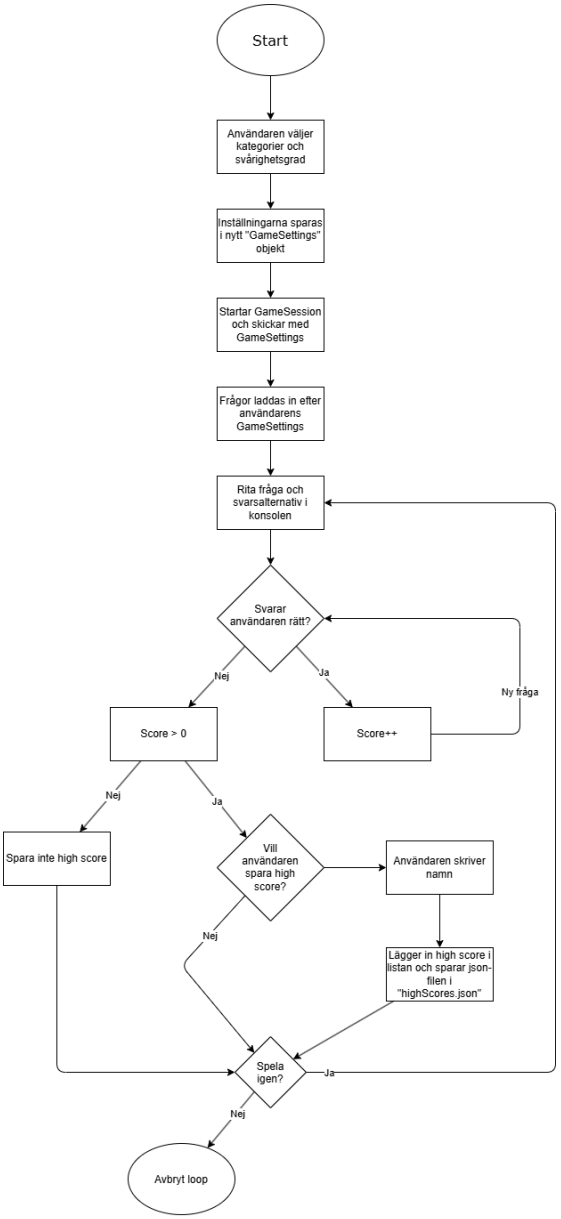
- **IQuestionLoader:** Definierar kontraktet för hur frågor läses in till applikationen. Genom att bygga mot detta gränssnitt kan inläsningen ske från exempelvis en JSON-fil (JsonQuestionLoader) utan att resten av koden behöver känna till varifrån datan kommer.

- **IHighScoreStorage:** Hanterar sparande och hämtning av de sparade resultaten. På samma sätt kan resultaten serialiseras och deserialiseras inför läsning eller sparande, vilket gör det enkelt att i framtiden byta ut filhanteringen mot exempelvis en databas.

6.3 Spelflöde & logik

För att styra hur spelsessionen exekveras och hur användaren navigerar genom spelet implementerades två huvudsakliga tjänster. **QuizService** ansvarar för inläsningen av kategorier och frågor, vilket låter användaren skräddarsy inställningarna för sin spelsession så att enbart relevanta frågor läses in. Därefter tar tjänsten **GameSession** vid och hanterar den aktiva spelomgången. Här styrs det löpande flödet där frågor presenteras, användarens svar valideras, poäng räknas samman och tillhörande logik exekveras.

Spelsessionens övergripande flöde, från uppstart till avslutad spelomgång och sparande av high score, illustreras i flödesschemat nedan **(se Figur 3)**.



Figur 3: Flödesschema som visar spelets logiska gång och spelflöde.

För att ge en dynamisk spelupplevelse där frågor eller svarsalternativ inte alltid presenteras i samma ordning används LINQ. Ett temporärt slumpmässigt ID skapas för varje svarsalternativ under sortering och genom att sedan sortera baserat på detta id uppnås en effektiv slumpning direkt i koden enligt kodexemplet nedan **(se Figur 4)**:

```
return decodedAnswers
        .OrderBy(_ ⇒ Guid.NewGuid())
        .ToList();
```

Figur 4: Kodexempel på slumpning med LINQ och Guid.

6.4 Användargränssnitt och navigation

Eftersom applikationen är utvecklad som ett konsolprogram lades stort fokus på att skapa ett interaktivt och modernt gränssnitt direkt i terminalen, utan krav på manuell textinmatning för menyval. För att uppnå detta och höja användarupplevelsen gjordes navigationen samt utskrift av text och tabeller med hjälp av det externa biblioteket *Spectre.Console*.

Gränssnittet är uppbyggt i olika vyer som hanterar spelets olika delar. För att hålla koden DRY (Don’t Repeat Yourself) och säkerställa ett sammanhängande utseende skapades tjänsten *HeaderPanel.cs*, vilken skriver ut en gemensam rubrik och avdelare som återanvänds i varje vy. Följande utskrift visar komponenten med en inparameter för texten i avdelaren **(se Figur 5)**:



Figur 5: Panel som används genomgående i applikationen för ett sammanhängande gränssnitt.

Navigationen var ett av projektets största fokusområden för att helt undvika manuella inmatningar. Med hjälp av *Spectre.Console* och dess *SelectionPrompt* samt *MultiSelectionPrompt* möjliggjordes förgjorda menyalternativ som styrs smidigt med piltangenter och Enter, istället för att användaren exempelvis behöver skriva in en siffra. Nedan visas ett exempel på en *MultiSelectionPrompt* för flerval **(se Figur 6)**:

```
var prompt = new MultiSelectionPrompt<string>()
  .AddChoices(allCategories)
  .HighlightStyle(new Style(foreground: Color.Yellow));
```

Figur 6: *MultiSelectionPrompt* för att kunna markera flera kategorier med tangentbordsnavigering.

7 Resultat

Projektet resulterade i en fullt fungerande, robust och modern konsolapplikation för ett quizspel skrivet i C# .NET. För att utvärdera hur väl projektet uppfyller de ursprungliga målsättningarna följer nedan en sammanställning kopplad till rapportens fyra centrala frågeställningar:

- **Modularitet:** Genom uppdelningen av koden i tydliga mappar (Models, Services, Interfaces, Screens, Utils) och tillämpning av *Dependency Injection* via gränssnitten *IQuestionLoader* och *IHighScoreStorage*, separerades logiken från både användargränssnitt och datakällor. Detta säkerställde att applikationen kan hantera ändringar i exempelvis hur data lagras utan att påverka resten av kodens flöde.

- **Användarupplevelse:** Det terminalbaserade gränssnittet har med framgång gått från traditionella, manuella siffer- och textinmatningar till en modern och engagerande upplevelse. Integreringen av *Spectre.Console* tillät tangentbordsnavigation, och tillsammans med återanvändbara paneler (*HeaderPanel.cs*) kan applikationen presenteras på ett tydligt och sammanhängande sätt.

- **Extern datahantering:** Extern data från JSON-filer hanteras på ett kontrollerat sätt via serialisering och deserialisering med *System.Text.Json*. Ifall inläsning skulle misslyckas laddas en tom lista in i stället för att undvika att programmet kraschar vid uppstart. Genom att filtrera den inlästa datan utifrån användarens val av kategorier och svårighetsgrad säkerställdes en dynamisk datahantering som endast presenterar det användaren vill se.

- **Spelflöde och validering:** Hela applikationen är uppbyggd kring en sluten loop som endast kan avbrytas via aktiv bekräftelse från användaren. Även de inre looparna är slutna där användaren väljer när spelet ska avbrytas och menyn ska visas igen. Detta ger en robust applikation som känns mer som ett modernt spel än ett enkelt program som körs igenom en enda gång. Eftersom användaren styrs via menyer och inte kan ange fria textinmatningar krävdes ingen omfattande validering i körtid för användarinput, istället skedde valideringen i bakgrunden vid datainläsning, exempelvis genom att kontrollera att filerna som ska läsas in existerar.

8 Slutsatser

8.1 Reflektion kring resultat

Projektet har framgångsrikt resulterat i en fullt fungerande och robust konsolapplikation som uppfyller samtliga uppställda mål från inledningen. Genom att konsekvent tillämpa objektorienterade och SOLID-principer skapades en struktur som inte enbart uppfyller projektkraven utan även är lätt att underhålla och bygga ut. Övergången från traditionella, manuella textinmatningar höjde helhetsintrycket avsevärt. Resultatet visar att C# .NET lämpar sig mycket bra för att bygga strukturerade, moderna och interaktiva konsolapplikationer som efterliknar mer moderna och avancerade programvaror.

8.2 Lärdomar och utveckling

Under projektet har följande värdefulla lärdomar dragits:

- **Filstruktur och separation av ansvar:** Vikten av att tidigt dela upp koden i tydliga ansvarsområden med en logisk filstruktur blev tydlig. Detta arbetssätt underlättade felsökning och gjorde att ändringar och tillägg i koden skedde sömlöst.

- **Externa bibliotek:** Att arbeta med *Spectre.Console* gav en djupare förståelse för hur viktigt det är att inte låsa fast sig i en specifik programvara eller funktionalitet. I stället bör stor vikt läggas vid att utforska nya ramverk och bibliotek för att förbättra sitt projekt.

- **Objektorienterad programmering:** Innan projektet var erfarenheterna inom objektorienterad programmering minimala då endast några få tidigare projekt använt sig av detta arbetssätt. I detta projekt fick dessa principer tillämpas mer utförligt, vilket har lett till en stor förbättring och värdefull erfarenhet av att arbeta objektorienterat.

8.3 Framtida utveckling

Trots att applikationen fyller sitt syfte och fungerar väl finns det flera aspekter som skulle kunna förbättras och vidareutvecklas i framtiden:

- **Databasintegration:** Tack vare att applikationen är uppbyggd med gränssnitt är den förberedd för att byta ut lagringen med JSON-filerna till en mer gedigen och riktig databas, exempelvis SQLite.

- **Fler spelvarianter:** Applikationen stödjer enbart ett spelläge där spelaren har ett liv och ska försöka få rätt på så många frågor i rad som möjligt. Vidareutveckling hade kunnat ge användaren möjlighet att aktivera ett flerlivssystem, tidsbegränsade frågor eller ett flerspelarläge som spelas i turordning.

- **Utökad frågehantering:** Möjlighet att lägga till ett administratörsläge där applikationens frågor och kategorier kan redigeras, tas bort och läggas till

direkt via terminalen, i stället för att användaren manuellt behöver ändra i JSON-filen.

9 Källförteckning

[1] Microsoft Learn. En rundtur i C#-språket [Internet]. Redmond (WA): Microsoft; 2026 [uppdaterad 2026 feb 24; hämtad 2026 nov 1]. Tillgänglig från: https://learn.microsoft.com/sv-se/dotnet/csharp/tour-of-csharp/overview

[2] GeeksforGeeks. SOLID Principle in Programming: Understand with Real Life Examples [Internet]. 2023 [hämtad 2026 nov 1]. Tillgänglig från: https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/

[3] Oracle. What Is JSON? [Internet]. Oracle Sverige; 2024 [hämtad 2026 nov 1]. Tillgänglig från: https://www.oracle.com/se/database/what-is-json/

[4] Advania. CLI: Vad är Command Line Interface och hur används det? [Internet]. Ordförrådet; [hämtad 2026 nov 1]. Tillgänglig från: https://www.advania.se/ordforradet/cli

[5] .NET Landscape. Spectre.Console – Terminal UI for .NET [Internet]. 2026 [hämtad 2026 nov 1]. Tillgänglig från: https://dotnetlandscape.com/l/spectre-console/

[6] GeeksforGeeks. LINQ (Language Integrated Query) [Internet]. 2026 [uppdaterad 2026 okt 29; hämtad 2026 nov 1]. Tillgänglig från: https://www.geeksforgeeks.org/c-sharp/linq-language-integrated-query/

[7] Microsoft Learn. HttpUtility.HtmlDecode Method (System.Web) [Internet]. Redmond (WA): Microsoft; 2026 [hämtad 2026 nov 1]. Tillgänglig från: https://learn.microsoft.com/en-us/dotnet/api/system.web.httputility.htmldecode?view=net-10.0